Data Structures and Algorithms — Lab 6
Trees and Tree Variants

Hierarchical Data Structures and Recursive Thinking

School of Computer Science and Engineering

Lab Theme. In previous labs, you studied linear and order-based data structures such as Stack, Queue, Priority Queue, and Hash Table. Those structures organize data using rules such as LIFO, FIFO, priority order, or key-based lookup.
In this lab, you will study **Trees**, which organize data in a **hierarchical structure**. Trees are widely used in file systems, decision making, expression evaluation, search engines, database indexing, compilers, autocomplete systems, and many algorithmic problems.
The key question for this lab is:

How should data be organized hierarchically so that searching, traversal, ranking, or range operations become efficient?

1 Learning Outcomes

After completing this lab, students should be able to:

- explain the basic terminology of trees;
- distinguish between general trees, binary trees, binary search trees, heaps, tries, and balanced trees;
- implement a binary tree node structure in Java;
- perform tree traversals using preorder, inorder, postorder, and level-order traversal;
- explain the difference between DFS and BFS on trees;
- implement insertion and search in a Binary Search Tree;
- explain why BST performance depends on tree height;
- explain the idea of balanced trees such as AVL Tree and Red-Black Tree;
- explain why Heap is a tree-based structure used for Priority Queue;
- explain the use of Trie for prefix-based string search;
- identify suitable tree variants for different problems;
- analyze time complexity and space complexity of tree operations.

2 General Requirements

2.1 Programming Requirements

- Use **Java**.
- Organize source code by tasks or exercises.
- Use meaningful class names, method names, and variable names.
- Your program must compile and run using standard Java commands.
- You may use Java built-in classes only when the exercise allows it.
- For manual implementation exercises, you must define your own node structure.

Useful Java classes may include:

```
import java.util.Queue;
import java.util.LinkedList;
import java.util.ArrayList;
import java.util.List;
import java.util.Stack;
import java.util.PriorityQueue;
import java.util.HashMap;
```

However, for the required Binary Tree, Binary Search Tree, Min Heap, and Trie exercises, students must implement the core structure manually.

2.2 Problem-Solving and Explanation Requirements

For each required exercise, your report should include:

1. problem restatement;
2. input, output, assumptions, and edge cases;
3. selected tree structure;
4. what information is stored in each node;
5. how nodes are connected;
6. algorithm idea;
7. why the selected tree structure is suitable;
8. time complexity and space complexity;
9. at least **3 test cases**.

During explanation, do not explain the code line by line. Instead, focus on the idea.

Ask yourself:

- What does each node store?
- What does the left child represent?
- What does the right child represent?
- What is the root node?
- What is the base case for recursion?
- Is the tree balanced or unbalanced?
- What determines the height of the tree?
- Why does the tree improve search, traversal, or organization?

3 Core Concept 1: Tree Basics

3.1 Definition

A **Tree** is a hierarchical data structure made of nodes connected by edges.

A tree has:

- one **root** node;
- zero or more **child** nodes;
- no cycles;
- exactly one path from the root to any node.

Example:

```
        A
      / | \
     B  C  D
    / \
   E   F
```

In this example:

- `A` is the root;
- `B`, `C`, and `D` are children of `A`;

- `A` is the parent of `B`, `C`, and `D`;
- `E` and `F` are leaf nodes;
- the tree height depends on the longest path from the root to a leaf.

3.2 Important Tree Terminology

Term	Meaning
Root	The top node of a tree
Parent	A node that has child nodes
Child	A node connected below another node
Sibling	Nodes with the same parent
Leaf	A node with no children
Edge	A connection between two nodes
Path	A sequence of connected nodes
Depth	Number of edges from root to a node
Height	Number of edges on the longest path from a node to a leaf
Subtree	A tree formed from a node and its descendants

4 Core Concept 2: Common Tree Variants

Trees have many variants. Each variant is designed for a different purpose.

Tree Variant	Main Idea	Common Use
General Tree	A node may have any number of children	File systems, organization charts
Binary Tree	Each node has at most two children	Traversal, expression trees
Binary Search Tree	Left child values are smaller, right child values are larger	Searching, sorted data
AVL Tree	Self-balancing BST using rotations	Fast search with guaranteed height balance
Red-Black Tree	Self-balancing BST using color rules	Java TreeMap, TreeSet, database-like structures
Heap	Complete binary tree with heap order property	Priority Queue, scheduling, top-k problems
Trie	Tree for storing strings character by character	Autocomplete, dictionary, prefix search
Segment Tree	Tree for answering range queries	Range sum, range minimum, range maximum

Fenwick Tree	Compact tree-like structure for prefix sums	Efficient cumulative frequency/range sum
B-Tree	Multi-way balanced search tree	Database and file system indexing

For this one-week lab, the required implementation focus is:

- Binary Tree traversal;
- Binary Search Tree operations;
- Heap concept and usage;
- Trie implementation.

Other variants are introduced conceptually so students can recognize them in future topics.

5 Core Concept 3: Binary Tree

5.1 Definition

A **Binary Tree** is a tree where each node has at most two children:

- left child;
- right child.

Example:

```
        10
       /  \
      5    20
     / \     \
    3   7     30
```

A simple Java node can be represented as:

```java
class TreeNode {
    int value;
    TreeNode left;
    TreeNode right;

    TreeNode(int value) {
        this.value = value;
        this.left = null;
        this.right = null;
    }
}
```

5.2 Tree Traversal

Tree traversal means visiting all nodes in a specific order.

There are two major traversal families:

1. **Depth-First Search**, also called DFS;
2. **Breadth-First Search**, also called BFS.

5.3 DFS Traversals

DFS goes deep into a branch before moving to another branch.

5.3.1 Preorder Traversal

Order:

```
Root -> Left -> Right
```

Useful when copying a tree or printing tree structure.

5.3.2 Inorder Traversal

Order:

```
Left -> Root -> Right
```

For a Binary Search Tree, inorder traversal gives values in sorted order.

5.3.3 Postorder Traversal

Order:

```
Left -> Right -> Root
```

Useful when deleting a tree or evaluating expression trees.

5.4 BFS Traversal: Level Order

BFS visits nodes level by level.

Example:

```
        10
       /  \
      5    20
```

```
    / \      \
   3   7      30
```

Level-order traversal:

```
10 5 20 3 7 30
```

A queue is usually used for level-order traversal.

6 Core Concept 4: Binary Search Tree

6.1 Definition

A **Binary Search Tree**, or **BST**, is a binary tree with the following rule:

For every node:

- all values in the left subtree are smaller than the node value;
- all values in the right subtree are larger than the node value.

Example:

```
        50
       /  \
     30    70
    / \    / \
   20 40  60 80
```

6.2 BST Search

To search for a value:

- start at the root;
- if target equals current node, found;
- if target is smaller, go left;
- if target is larger, go right.

This is faster than scanning all nodes when the tree is balanced.

6.3 BST Insertion

To insert a value:

- start at the root;
- compare the new value with the current node;
- go left if smaller;
- go right if larger;
- insert when an empty position is found.

6.4 BST Complexity

Operation	Balanced BST	Unbalanced BST
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Inorder traversal	$O(n)$	$O(n)$

The performance of a BST depends heavily on its height.

A balanced tree has small height. An unbalanced tree may become similar to a linked list.

Example of an unbalanced BST:

```
10
  \
   20
     \
      30
        \
         40
```

7 Core Concept 5: Balanced Trees

7.1 Why Balance Matters

A BST is efficient only when its height is small.

If values are inserted in sorted order, the BST can become unbalanced. Balanced trees solve this problem by keeping the height close to $\log n$.

7.2 AVL Tree

An **AVL Tree** is a self-balancing Binary Search Tree.

For every node:

```
balance factor = height(left subtree) - height(right subtree)
```

The balance factor must be:

```
-1, 0, or 1
```

If the tree becomes unbalanced, rotations are used.

Common rotations:

- Left Rotation;
- Right Rotation;
- Left-Right Rotation;
- Right-Left Rotation.

In this lab, students only need to understand the idea. Full AVL implementation is further practice.

7.3 Red-Black Tree

A **Red-Black Tree** is another self-balancing BST.

It uses color rules to maintain balance.

Java uses Red-Black Tree ideas in structures such as:

```
TreeMap
TreeSet
```

Students are not required to implement Red-Black Tree in this lab.

8 Core Concept 6: Heap

8.1 Definition

A **Heap** is a complete binary tree that satisfies a heap property.

There are two common types:

Heap Type	Rule
Min Heap	Parent value is smaller than or equal to child values
Max Heap	Parent value is greater than or equal to child values

Example Min Heap:

```
        2
       / \
      5   8
     / \  /
    10 12 9
```

The smallest value is always at the root.

8.2 Heap and Priority Queue

A Priority Queue is commonly implemented using a Heap.

In Java:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

This creates a min-priority queue by default.

For max-priority:

```
PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> b - a);
```

Heap is useful when we repeatedly need:

- smallest item;
- largest item;
- top k items;
- next task with highest priority.

9 Core Concept 7: Trie

9.1 Definition

A **Trie**, also called a prefix tree, is used to store strings character by character.

Example words:

```
cat
car
```

```
care
dog
```

Trie structure:

```
root
 |- c
 |   `- a
 |      |- t
 |      `- r
 |         `- e
 `- d
    `- o
       `- g
```

9.2 Trie Node

A simple Trie node may store:

- child links;
- a boolean value indicating whether a word ends at this node.

Example:

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord = false;
}
```

9.3 Trie Operations

Common operations:

```
insert(word)
search(word)
startsWith(prefix)
```

Trie is useful for:

- autocomplete;
- dictionary lookup;
- spell checking;
- prefix search;

- word games.

If the word length is L, Trie operations usually take:

```
O(L)
```

10 Core Concept 8: Segment Tree and Fenwick Tree

10.1 Segment Tree

A **Segment Tree** is used for range queries.

Example problems:

- find the sum of values from index l to index r;
- find the minimum value in a range;
- find the maximum value in a range;
- update one value and answer future queries efficiently.

Typical complexities:

Operation	Complexity
Build	$O(n)$
Range query	$O(\log n)$
Point update	$O(\log n)$

Segment Tree is useful when the array changes and many range queries are needed.

10.2 Fenwick Tree

A **Fenwick Tree**, also called Binary Indexed Tree, is a compact structure for prefix sums.

Typical complexities:

Operation	Complexity
Update	$O(\log n)$
Prefix sum query	$O(\log n)$
Range sum query	$O(\log n)$

Fenwick Tree is usually easier to implement than Segment Tree for sum-based problems.

For this lab, Segment Tree and Fenwick Tree are introduced conceptually only.

11 Comparison of Tree Variants

Structure	Key Rule	Best Used When
Binary Tree	Each node has at most two children	Traversal, recursive thinking
BST	Left smaller, right larger	Search, sorted data
AVL Tree	BST with strict height balance	Guaranteed fast search
Red-Black Tree	BST with color-based balance	Library-level ordered maps/sets
Heap	Parent has higher/lower priority than children	Priority Queue, top-k
Trie	Each path represents characters of a word	Prefix search, autocomplete
Segment Tree	Each node stores information about a range	Range queries with updates
Fenwick Tree	Stores partial prefix information	Prefix/range sum with updates
B-Tree	Multi-way balanced tree	Database/file-system indexing

12 Part A: Guided Practice Questions

These tasks are designed to strengthen tree selection and explanation skills before implementation.

12.1 Task A1: Identify the Best Tree Variant

For each problem, identify the most suitable structure: Binary Tree, BST, AVL Tree, Heap, Trie, Segment Tree, Fenwick Tree, B-Tree, HashMap, or another structure. Briefly justify your answer.

1. Print all values of a tree level by level.
2. Store numbers so that searching can be faster than linear search.
3. Store dictionary words and check whether a prefix exists.
4. Repeatedly retrieve the smallest task priority.
5. Answer many range-sum queries on an array with updates.
6. Store database index keys efficiently on disk.

7. Print BST values in sorted order.
8. Keep a search tree balanced after many insertions.
9. Build an autocomplete feature for a search box.
10. Find the kth largest element in an array.

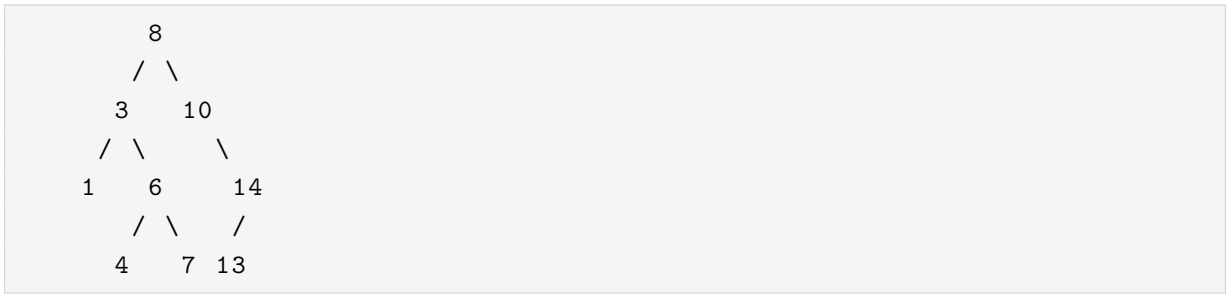
12.2 Task A2: What Is Stored?

Choose 5 problems from Task A1. For each problem, explain:

- what data structure is used;
- what each node stores;
- how nodes are connected;
- what operation needs to be efficient;
- why this structure is suitable.

12.3 Task A3: Traversal Recognition

Given the tree below:



Write the output of:

1. preorder traversal;
2. inorder traversal;
3. postorder traversal;
4. level-order traversal.

Then answer:

- Which traversal gives sorted order for this BST?
- Which traversal is most suitable for level-by-level printing?
- Which traversal is useful if we want to delete nodes from bottom to top?

13 Part B: Required Exercises

Complete all required exercises below.

For each exercise, include:

- problem restatement;
- selected tree structure;
- explanation of what is stored;
- algorithm idea;
- complexity analysis;
- at least 3 test cases.

13.1 Exercise B1: Binary Tree Traversals

Question. Given a binary tree, implement the following traversals:

```
preorder(root)
inorder(root)
postorder(root)
levelOrder(root)
```

Use this sample tree:

```
        10
       /  \
      5    20
     / \     \
    3   7     30
```

Expected outputs:

```
Preorder: 10 5 3 7 20 30
Inorder: 3 5 7 10 20 30
Postorder: 3 7 5 30 20 10
Level-order: 10 5 20 3 7 30
```

Focus. Binary Tree traversal, DFS, BFS.

Students must explain:

- What does each node store?
- What is the difference between preorder, inorder, and postorder?

- Why does level-order traversal use a queue?
- What is the base case for recursive traversal?
- What is the time complexity?

13.2 Exercise B2: Binary Search Tree Insert and Search

Question. Implement a Binary Search Tree that supports:

```
insert(int value)
search(int value)
inorderTraversal()
```

Example:

```
Insert: 50, 30, 70, 20, 40, 60, 80
Search 60 -> true
Search 25 -> false
Inorder -> 20 30 40 50 60 70 80
```

Focus. BST property and searching.

Required behavior:

- If a value is smaller than the current node, go left.
- If a value is larger than the current node, go right.
- You may ignore duplicate values or count duplicates, but you must clearly state your choice.

Students must explain:

- What is the BST property?
- What does the left subtree contain?
- What does the right subtree contain?
- Why can BST search be faster than linear search?
- When does BST search become slow?
- What is the complexity in balanced and unbalanced cases?

13.3 Exercise B3: Validate a Binary Search Tree

Question. Given the root of a binary tree, determine whether it is a valid Binary Search Tree.

A valid BST must satisfy:

- all nodes in the left subtree are smaller than the current node;
- all nodes in the right subtree are larger than the current node;
- both left and right subtrees must also be valid BSTs.

Example 1:

```
Input:
        5
       / \
      3   7

Output: true
```

Example 2:

```
Input:
        5
       / \
      3   7
         /
        4

Output: false
```

Explanation: `4` is in the right subtree of `5`, but it is smaller than `5`.

Focus. BST validation using lower and upper bounds.

Hint: Each node should be checked using a valid range:

```
min < node.value < max
```

When moving left, update the upper bound. When moving right, update the lower bound.

Students must explain:

- Why is checking only the direct left and right child not enough?
- What do the lower and upper bounds mean?
- How does recursion help validate the whole tree?
- What is the time complexity?

13.4 Exercise B4: Implement a Min Heap Using an Array

Question. Implement a Min Heap using an array or ArrayList.

Your class should support:

```
insert(int value)
peekMin()
extractMin()
isEmpty()
```

Example:

```
insert(10)
insert(4)
insert(15)
insert(2)
peekMin() -> 2
extractMin() -> 2
peekMin() -> 4
```

Focus. Heap structure and heap property.

Required behavior:

For a node at index `i`:

```
left child index = 2 * i + 1
right child index = 2 * i + 2
parent index = (i - 1) / 2
```

After insertion, use **heapify up**. After extraction, use **heapify down**.

Students must explain:

- Why can a heap be stored in an array?
- What is the Min Heap property?
- What is heapify up?
- What is heapify down?
- Why is `extractMin` $O(\log n)$?
- How is Heap related to Priority Queue?

13.5 Exercise B5: Implement a Trie for Word Search

Question. Implement a Trie that supports:

```
insert(String word)
search(String word)
startsWith(String prefix)
```

Example:

```
insert("cat")
insert("car")
insert("care")
search("car") -> true
search("cap") -> false
startsWith("ca") -> true
startsWith("dog") -> false
```

Focus. Trie and prefix search.

Required assumptions:

- Words contain lowercase English letters `a` to `z`.
- Each Trie node may store an array of 26 child references.
- `isEndOfWord` is used to mark a complete word.

Students must explain:

- What does each Trie node store?
- Why do we need `isEndOfWord`?
- What is the difference between `search` and `startsWith`?
- Why is Trie efficient for prefix search?
- What is the time complexity in terms of word length?

14 Part C: Further Practice Questions

Choose **3 out of 6** problems below.

For each selected problem, include:

1. problem restatement;
2. selected tree structure;
3. explanation of what is stored;
4. algorithm idea;
5. time and space complexity;
6. at least 3 test cases.

14.1 C1: Maximum Depth of a Binary Tree

Question. Given the root of a binary tree, return its maximum depth.

The maximum depth is the number of nodes along the longest path from the root node down to a leaf node.

Example:

```
Input:
        3
       / \
      9   20
         /  \
        15   7

Output: 3
```

Focus. Binary Tree recursion.

Hint:

```
1 + max(depth(left), depth(right))
```

Students must explain:

- What is the base case?
- Why do we take the maximum of left and right depth?
- What is the time complexity?

14.2 C2: Lowest Common Ancestor in a BST

Question. Given a BST and two values `p` and `q`, find their lowest common ancestor.

The lowest common ancestor is the lowest node that has both `p` and `q` as descendants.

Example:

```
Input BST:
        6
       / \
      2   8
     / \ / \
    0  4 7  9
      / \
     3   5

p = 2, q = 8
Output: 6
```

Focus. Using BST property.

Hint:

- If both values are smaller than current node, go left.
- If both values are larger than current node, go right.
- Otherwise, current node is the lowest common ancestor.

14.3 C3: Kth Smallest Element in a BST

Question. Given a BST and an integer `k`, return the kth smallest value.

Example:

```
Input BST:
         5
        / \
       3   6
      / \
     2   4
    /
   1

k = 3
Output: 3
```

Focus. BST inorder traversal.

Hint: Inorder traversal of a BST gives values in sorted order.

Students must explain:

- Why does inorder traversal produce sorted order?
- How do we count visited nodes?
- What is the time complexity?

14.4 C4: Merge Two Binary Trees

Question. Given two binary trees, merge them into one tree.

If two nodes overlap, the new value is the sum of both node values. If only one node exists, use that node.

Example:

```
Tree 1:
      1
```

```
     / \
    3   2
   /
  5

Tree 2:
      2
     / \
    1   3
     \   \
      4   7

Merged:
      3
     / \
    4   5
   / \   \
  5   4   7
```

Focus. Recursive tree construction.

14.5 C5: Autocomplete Using Trie

Question. Given a list of words and a prefix, return all words that start with the prefix.

Example:

```
Words: ["cat", "car", "care", "dog", "door"]
Prefix: "ca"
Output: ["cat", "car", "care"]
```

Focus. Trie traversal and DFS.

Hint: First move to the Trie node representing the prefix. Then use DFS to collect all complete words below that node.

14.6 C6: Range Sum Query Using Segment Tree

Question. Given an integer array, build a Segment Tree that can answer range sum queries.

Example:

```
Array: [1, 3, 5, 7, 9, 11]
Query sum(1, 3) -> 15
```

Explanation:

```
nums[1] + nums[2] + nums[3] = 3 + 5 + 7 = 15
```

Focus. Segment Tree concept.

This is an advanced practice question. Students may use this as preparation for future labs.

15 Part D: Reflection and Explanation

15.1 Task D1: Concept Reflection

Write a short reflection answering:

1. What is the difference between a linear data structure and a tree?
2. What is the difference between a Binary Tree and a Binary Search Tree?
3. Why does BST performance depend on height?
4. What happens when a BST becomes unbalanced?
5. Why does inorder traversal of a BST produce sorted values?
6. How is Heap different from BST?
7. How is Heap related to Priority Queue?
8. Why is Trie useful for prefix search?
9. What type of problem is Segment Tree designed for?
10. Which required exercise was the hardest, and why?

15.2 Task D2: Explain Without Code

Choose one required exercise from Part B and explain the solution in plain English without showing code.

Your explanation must include:

- the selected tree structure;
- what each node stores;
- how nodes are connected;
- when recursion or iteration is used;
- why the structure is suitable;
- why the complexity is efficient.

16 Testing Requirement

For every required exercise:

- include at least **3 test cases**;
- include at least **1 edge case**;
- show expected output and actual output;
- briefly explain why each test case is useful.

Suggested edge cases:

Exercise	**Possible Edge Case**
Binary Tree Traversals	Empty tree, single-node tree
BST Insert and Search	Searching for missing value, duplicate insertion
Validate BST	Invalid node deep inside subtree
Min Heap	Extract from empty heap, repeated values
Trie	Searching for prefix that is not a full word

17 Deliverables and Submission

Submit one `.zip` file containing:

Item	**Description**
Source code	Java files for all required exercises and selected further practice questions
Report PDF	Explanation, selected tree structure, stored information, algorithm idea, complexity analysis, and test evidence
Tests	Input/output screenshots or text files

18 Suggested Grading Breakdown

Criterion	**Weight**
Tree concept understanding and explanation	20%
Correct implementation of required exercises	30%
BST, Heap, and Trie reasoning	20%
Complexity analysis	15%
Code quality and organization	10%
Report clarity and test evidence	5%

19 Final Notes

Trees are not only another data structure. They represent a different way of thinking.

Instead of storing data in a simple sequence, trees organize data by relationship, hierarchy, priority, or prefix.

A strong solution should answer three questions:

1. What does each node store?
2. How are nodes connected?
3. Why does this tree structure make the operation efficient?

The most important goal is not only to write correct Java code, but also to explain why the chosen tree variant is appropriate for the problem.

A Suggested Java File Organization

Students may organize files as follows:

```
Lab6_Trees/
|
|-- BinaryTreeTraversal.java
|-- BinarySearchTree.java
|-- ValidateBST.java
|-- MinHeap.java
|-- Trie.java
|-- FurtherPractice1.java
|-- FurtherPractice2.java
‘-- README.txt
```

B Suggested Report Structure

For each exercise, use the following report template:

```
Exercise Title:

1. Problem Restatement
2. Input and Output
3. Selected Tree Structure
4. What Each Node Stores
5. Algorithm Idea
6. Why This Structure Is Suitable
7. Complexity Analysis
```

```
8. Test Cases
9. Short Reflection
```

C Short Concept Summary for Students

Problem Pattern	**Recommended Structure**
Need hierarchical representation	General Tree or Binary Tree
Need sorted traversal/search	BST
Need guaranteed balanced search	AVL Tree or Red-Black Tree
Need smallest/largest repeatedly	Heap / Priority Queue
Need prefix search	Trie
Need range query with updates	Segment Tree or Fenwick Tree
Need database/file indexing	B-Tree